

바이브 코딩으로 만든 우주 침략자 게임 실행 & 신경망

2021-17385

산업공학과 김예원

[1] 바이브 코딩으로 만든 우주 침략자 게임 실행

먼저 교수님이 Gemini와의 대화만으로 만든 우주 침략자 게임의 코드를 살펴보았다. 퀴즈 폴더에서 make를 실행하면 Gemini의 답변과 함께, 이전 버전과 Gemini가 "완벽하게 보완했다"고 한 새 버전의 코드를 diff로 비교할 수 있었다. 처음에는 빨간 줄이 가득해서 많은 부분이 바뀐 줄 알았다. 그런데 주석을 빼고 코드만 다시 비교해 보니, 실제로 바뀐 것은 출력 문구 몇 줄뿐이었고 동작하는 코드는 한 줄도 달라지지 않았다. 실제 기능 개선은 오히려 그 전 단계에서 이미 이루어져 있었다. AI가 자신 있게 "완벽하다"고 답변해도 직접 확인해 보지 않으면 알 수 없다는 것을 느꼈다. AI 시대에서의 사람의 역할이 AI 실행물을 검토하고 그에 대해 책임을 지는 방향이 될 것이라는 이야기를 들었는데, 여전히 사람의 역할이 크다는 것을 느낄 수 있었다.

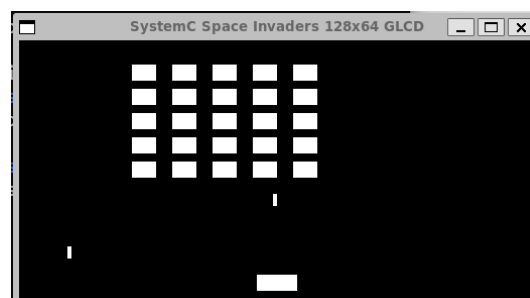
다음으로 Gemini가 준 Makefile로 게임을 빌드해 보았다. 이 과정에는 두 가지 도구가 쓰인다. Verilator는 Verilog로 된 게임 칩을 SystemC(C++) 코드로 바꿔 주는 도구이고, make는 Makefile에 적힌 순서대로 Verilator 변환과 C++ 컴파일을 자동으로 실행해 주는 도구이다. 하지만 make를 실행하자 Verilog 파일을 찾을 수 없다는 오류가 나며 멈췄다. 폴더를 확인해 보니 빌드에 필요한 Verilog 코드와 sc_main.cpp, 헤더 파일이 다른 폴더(Q09b)에 있었다. 오류 메시지와 diff 결과를 AI에게 보여 주고 원인과 수정 방법을 물었지만, 답을 그대로 쓰지 않고 폴더를 직접 확인하고 정상적으로 동작하는 Q09d의 Makefile과 비교하며 맞는지 확인했다. 그렇게 파일 경로, SystemC 설치 경로, --pins-sc-uint 옵션, 빌드 파일 이름 등 여섯 군데를 수정했다. 이때 Q09d를 그대로 베끼면 이전 버전의 키보드 코드(조이스틱 역할)로 빌드된다는 것을 알게 되어, 이번에 확인해야 할 Q09e의 파일을 쓰도록 직접 바꿨다. 수정하는 과정에서 uint를 unit으로 잘못 쓰거나 띄어쓰기를 빠뜨려 두 경로가 붙어버리는 실수도 있었는데, Verilator가 오류 메시지에 올바른 옵션 이름을 제안해 주어 쉽게 고칠 수 있었다.

```
Build & Run? [Y/n]:
make[1]: Entering directory '/home/yewon/ETRI-0.5um-CMOS-MPW-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz'
[BUILD STAGE 1] Verilator 가동 -> Verilog 하드웨어를 SystemC 3.0 C++ 소스로
코딩 이식 복사
verilator --sc --exe --Wall --no-fatal \
-I. \
sc_main.cpp sc_glcd128x64_TLM.cpp sc_invaders_TB.cpp space_invaders_
engine_glcd.v \
-DFLAGS "-L/usr/local/lib -Wl,-rpath,/usr/local/lib -Wl,--enable-ne
w-dtags -lSDL2 -L/usr/local/systemc-3.0.0/lib-linux64 -lsystemc" \
-CFLAGS "-std=c++17 -I/usr/local/include/SDL2 -D_REENTRANT -I/usr/lo
cal/systemc-3.0.0/include" \
-Mdir obj_dir
%Error: Cannot find file containing module: 'space_invaders_engine_glcd.v'
... See the manual at https://verilator.org/verilator_doc.html?v=5.0
49 for more assistance.
... Looked in:
space_invaders_engine_glcd.v
space_invaders_engine_glcd.v.v
space_invaders_engine_glcd.v.v
obj_dir/space_invaders_engine_glcd.v
obj_dir/space_invaders_engine_glcd.v.v
obj_dir/space_invaders_engine_glcd.v.v
... With current working directory /home/yewon/ETRI-0.5um-CMOS-MPW-
Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_
Quiz'
%Error: Exiting due to 1 error(s)
make[1]: *** [Makefile.mk:24: compile_and_run] Error 1
make[1]: Leaving directory '/home/yewon/ETRI-0.5um-CMOS-MPW-Std-Cell-DK/Proj
ects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz'
make: *** [Makefile:21: all] Error 2
```

Makefile.mk에서 수정한 사항

#	수정위치	원래 (Gemini)	수정 후	이유
1	SystemC 경로	SYSTEMC_HOME ?= /usr/local/...	export SYSTEMC=/opt/systemc 등 5줄	이 PC의 실제 설 치 위치
2	RTL_SRC	space_invaders_engine _glcd.v	../Q.09b_.../space_invaders_ engine_glcd.v	Verilog 파일이 Q.09b에 있음
3	CPP_SRC	sc_main.cpp (경로 없 음)	../Q.09b_.../sc_main.cpp	sc_main.cpp가 Q.09b에 있음
4	Verilator 옵션	없음	--pins-sc-uint 추가	포트 타입을 sc_uint로 맞춤
5	-CFLAGS	헤더 경로 없음	-I../..../Q.09b_... 추가	헤더(.h)가 Q.09b 에 있음
6	빌드 파 일 이름	Vspace_invaders_engin e.mk	Vspace_invaders_engine_glc d.mk	최상위 모듈 이 름을 따름

빌드에 성공하고 실행시켜보았으나, 이번에는 게임 창이 보이지 않았다. 작업 표시줄에 아이콘만 생기고 화면은 뜨지 않았다. 처음에는 128×64 픽셀로 만들어진 창이 너무 작아서라고 생각해, 창 크기를 512×256으로 키우도록 코드 한 줄을 수정했다. 그래도 창이 뜨지 않아 간단한 그래픽 프로그램인 glxgears를 실행해 보니 이것도 보이지 않았다. 게임이 아니라 WSL의 화면 표시 기능이 문제였던 것이다. WSL을 재시작한 뒤 다시 실행하자 5×5로 늘어선 외계인과 대포, 포탄이 움직이는 게임 화면을 볼 수 있었다. 문제를 게임 코드의 문제와 실행 환경의 문제로 나눠서 확인하니 원인을 빨리 찾을 수 있었다. 마지막으로 퀴즈용 make를 다시 실행하자, 처음에 오류가 났던 Build & Run 단계가 성공하며 퀴즈를 통과했다. 이때 이미 빌드된 파일은 다시 컴파일하지 않고 "up to date"라고 넘어가는 것을 보며, make가 바뀐 파일만 골라 다시 빌드한다는 것도 알 수 있었다.



이번 과제를 하며 바이트 코딩이 편리하지만 한계도 분명하다는 것을 느꼈다. Gemini는 "완벽한" 코드와 Makefile을 주었지만, 내 컴퓨터의 파일 위치나 설치 경로, 실행 환경을 알지 못해 그대로는 동작하지 않았다. 결국 오류 메시지를 읽고, 정상 동작하는 코드와 비교하고, 무엇을 가져오고 무엇을 바꿀지 판단하는 일은 사람의 몫이었다. 특히 칩은 한 번 제작하면 고칠 수 없기 때문에, AI가 만든 결과를 그대로 믿기보다 직접 검증하는 과정이 더 중요하다고 생각했다. 지난번 탁구

게임기 코드를 한 줄씩 이해하려 했던 것처럼, AI를 활용하더라도 그 결과를 이해하고 검증할 수 있는 실력을 키워야겠다고 느꼈다.

[2] 신경망 이해 및 실습

신경망(NN)은 입력데이터를 보고 정답을 맞히도록 수많은 가중치를 조정하며 학습하는 계산 구조이다. 입력 x 가중치를 모두 더한 후 활성화 함수로 결과를 만든다.

신경망은 layer로 구성되는데, 해당 예시에서는 input (이미지 픽셀) $\rightarrow$ hidden layer $\rightarrow$ output (숫자 0~9 각각에 대한 점수, 가장 큰 값을 답으로 도출) 로 구성되어 있다. 학습 순서는 순전파 (forward propagation) - 손실(loss) 계산 - 역전파 (backpropagation) - 가중치 업데이트 - 반복 이다. 손실은 정답과 출력 값의 차이이고, 역전파는 이 오차를 가중치 비율대로 거꾸로 나눠 앞 layer의 오차를 구하는 과정이다. 오차가 줄어드는 방향으로 학습률만큼 가중치를 수정한다.

이론을 코드로 확인하기 위해 먼저 3×8 디코더를 학습하는 작은 신경망을 파이썬으로 실행해 보았다. 3비트 입력을 보고 8개의 출력 중 해당 번호 하나만 켜도록 학습시키는 것으로, 디지털 회로에서 배우는 디코더를 회로로 설계하는 대신 신경망이 예시를 보고 스스로 배우게 하는 실험 이다. 입력 3개, 은닉 노드 20개, 출력 8개, 학습률 0.3으로 신경망을 만들고 학습 전에 먼저 시험 해 보니, 가중치가 무작위 값이라 어떤 입력을 넣어도 4번 칸이 가장 크게 나와 입력을 전혀 구 별하지 못했다. 500번 학습시킨 뒤에는 8가지 입력 모두 정답 칸만 0.84~0.93으로 커지고 나머지는 0.1 이하로 작아졌다. 순전파, 오차 계산, 역전파, 가중치 업데이트를 반복하면서 무작위였던 가중치가 정답을 내는 값으로 바뀌는 과정을 직접 볼 수 있었다. 다만 입력이 모두 0일 때 정답 칸의 값(0.84)이 가장 낮았다. 입력이 0이면 가중치를 곱해도 0이 되어 신호가 잘 전달되지 않기 때문인데, 실습 코드에는 판단 기준(편향)이 없어서 생기는 한계였다. 편향이 왜 필요한지 결과로 확인할 수 있었다.

<pre>Input : [0. 0. 0.] Result: [[0.48343582] [0.37307533] [0.40415549] [0.25211196] [0.57898976] [0.24983643] [0.37250431] [0.48231353]] Input : [0.99 0. 0.] Result: [[0.48466155] [0.35537722] [0.45191353] [0.23959586] [0.61031374] [0.22812803] [0.40388174] [0.48992501]] Input : [0. 0.99 0.] Result: [[0.5077612] [0.37234648] [0.4052387] [0.26502556] [0.56217106] [0.23353676] [0.37300754] [0.45443205]]</pre>	<pre>Input : [0. 0. 0.] Result: [[0.84193415] [0.06134633] [0.05387476] [0.00455257] [0.04080097] [0.00988034] [0.00492661] [0.0020009]] Input : [0.99 0. 0.] Result: [[7.54239119e-02] [9.10051298e-01] [6.76634919e-04] [7.42207540e-02] [3.23394082e-04] [5.58288239e-02] [1.21635947e-05] [4.18161462e-03]] Input : [0. 0.99 0.] Result: [[7.12931401e-02] [3.86116756e-04] [9.14255437e-01] [7.06946613e-02] [4.51991617e-04] [8.53307777e-06] [5.25760987e-02] [2.09886293e-03]]</pre>
--	--

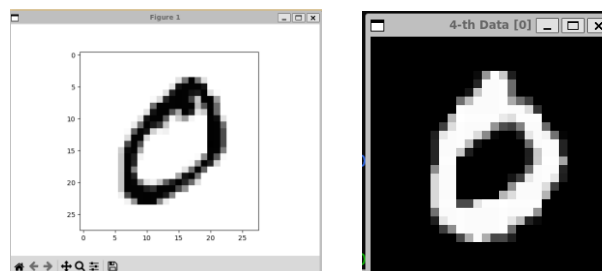
3×8 디코더 파이썬 결과 (왼쪽: 학습 전, 오른쪽: 500번 학습 후)

같은 실험을 C++로 해 보니 결과가 달랐다. 은닉층이 2개인 C++ 신경망은 100번 학습했을 때 8개 중 7개를 맞췄지만, 정답 칸의 값이 0.4~0.6 정도로 확신이 약했다. 학습 횟수를 파이썬과 같은 500번으로 늘리자 오히려 6개로 줄었고, 서로 다른 입력 세 개가 소수점까지 똑같은 출력을 내는 이상한 현상이 나타났다. 두 코드를 비교해 보니 C++에서는 첫 번째 은닉층의 값을 두 번째 은닉층으로 넘길 때 시그모이드를 거친 값이 아니라 거치기 전의 값을 넘기고 있었다. 학습이 진행될수록 이 값이 커져서 두 번째 은닉층의 출력이 0이나 1로 꼭 차 버리고(포화), 결국 입력을 구별하지 못하게 된 것이다. 원본을 복사해 이 한 줄을 고치자 포화 현상은 사라졌지만 정답은 4개로 더 줄었고, 입력 두 개를 한 쌍으로 묶어 반반씩 답하는 새로운 문제가 나타났다. 시그모이드의 미분을 계산할 때도 이미 시그모이드를 거친 출력에 시그모이드를 한 번 더 적용하는 차이가 남아 있었기 때문으로 보인다. 한 곳을 고쳤다고 바로 좋아지는 것이 아니라 여러 원인이 얹혀 있을 수 있다는 것을 직접 확인했다.

```
Test 3-to-8 Decoder:-----
Input : 0.01 0.01 0.01
Output: 0.24 0.03 0.00 0.26 0.00 0.14 0.01 [0.28]
Input : 0.99 0.00 0.00
Output: 0.01 [1.00] 0.00 0.01 0.00 0.01 0.00 0.00
Input : 0.00 0.99 0.00
Output: 0.03 0.00 [0.99] 0.03 0.00 0.00 0.14 0.02
Input : 0.99 0.99 0.00
Output: 0.24 0.03 0.00 0.26 0.00 0.14 0.01 [0.28]
Input : 0.00 0.00 0.99
Output: 0.04 0.01 0.00 0.02 [0.60] 0.29 0.16 0.05
Input : 0.99 0.00 0.99
Output: 0.17 0.06 0.01 0.22 0.14 [0.33] 0.01 0.20
Input : 0.00 0.99 0.99
Output: 0.00 0.00 0.00 0.00 0.01 0.00 [0.75] 0.00
Input : 0.99 0.99 0.99
Output: 0.24 0.03 0.00 0.26 0.00 0.14 0.01 [0.28]
End:-----
```

3×8 디코더 C++ 결과 (500번 학습, 000-011-111 입력의 출력이 똑같음)

다음으로 손글씨 숫자 데이터인 MNIST로 실습했다. 이미지 한 장은 가로 28, 세로 28인 784개의 화소로 이루어져 있고, CSV 파일 한 줄에서 첫 값이 정답 숫자, 나머지 784개가 화소의 밝기(0~255)이다. 신경망에 넣을 때는 이 밝기를 0.01~1.0 사이로 바꾸는데, 0을 피하는 이유도 3×8 디코더에서 본 것과 같았다. 같은 데이터를 파이썬은 흰 바탕에 검은 글씨로 한 번에 그렸고, C++는 검은 바탕에 흰 글씨로 방향키를 누를 때마다 화소 784개를 하나씩 그렸다. 학습용 10장으로 학습시킨 뒤 시험용 10장을 맞춰 보니 파이썬과 C++ 모두 정확도가 60%였다. 정확도는 같았지만 틀린 문제는 서로 달랐고, 둘 다 4와 9, 5와 4처럼 모양이 비슷한 숫자를 헷갈렸다. 사람이 봐도 애매한 글씨는 신경망의 확신도도 0.2~0.3으로 낮았다. 숫자 하나당 약 10장만 보고 배운 셈이라 한계가 있었고, 책에서는 6만 장 전체로 학습하면 95% 이상 나온다고 한다.



MNIST 손글씨 이미지 (왼쪽: 파이썬, 오른쪽: C++)

마지막으로 파이썬과 C++의 학습 코드를 비교했다. 두 코드는 순전파, 오차 계산, 역전파, 가중치 업데이트의 순서와 식이 같았다. 파이썬은 `numpy.dot` 한 줄로 행렬 곱을 하고 `.T`로 행렬을 뒤집어 오차를 거꾸로 보냈지만, C++는 이중 `for`문으로 곱하고 더하기를 하나씩 반복했고, 역전파에서는 가중치를 읽는 인덱스의 순서만 바뀌서 행렬을 뒤집은 효과를 냈다. 파이썬은 수식이 코드에 그대로 보여 이해하기 쉬웠고, C++는 계산 과정이 하나하나 드러났다. 특히 C++의 `oVector[i] += Wt[j][i] * iVector[j]` 한 줄이 곱셈과 누산을 반복하는 MAC 연산으로, MNIST에서 입력층에서 첫 번째 은닉층으로 한 번 넘어갈 때만 78,400번 실행된다. 신경망 칩은 이 연산을 수천 개씩 동시에 처리하는 회로라고 한다. 지난 과제에서 파이썬은 순서대로 처리하지만 Verilog는 병렬 회로로 동시에 실행할 수 있다는 것이 놀라웠는데, 신경망이야말로 이런 병렬 하드웨어가 필요한 계산이라는 것을 알 수 있었다.

파이썬과 C++ 신경망 코드 비교

비교 항목	파이썬	C++
층 구성 (MNIST)	784 → 200 → 10	784 → 100 → 100 → 10
행렬 곱	<code>numpy.dot()</code> 한 줄	<code>dot_product()</code> : 이중 <code>for</code> 문 (MAC 반복)
역전파 (전치)	<code>.T</code>	<code>dot_product_T()</code> : 인덱스 <code>[i][j]</code> 순서를 바꿈
시그모이드 미분	$o \times (1 - o)$	$\sigma(o) \times (1 - \sigma(o))$ (시그모이드 중복)
활성화 함수 선택	실행 중 (<code>lambda</code>)	빌드할 때 (<code>#if, -DSIGMOID / -DRELU</code>)
3×8 디코더 정답	8/8 (500번)	7/8 (100번), 6/8 (500번)
MNIST 정확도	60%	60%
장점	수식이 그대로 보여 이해하기 쉬움	곱셈·덧셈이 드러나 하드웨어로 옮기기 쉬움
단점	계산 과정이 <code>numpy</code> 안에 숨어 있음	코드가 길고 옮기는 과정에서 실수가 생기기 쉬움

AI가 만든 Makefile이 내 환경에서 그대로 돌아가지 않았던 것처럼, 같은 신경망 알고리즘도 C++로 옮기면서 생긴 한 줄의 차이가 결과를 바꿨다. 두 경우 모두 코드를 직접 비교하고 결과를 확인해야 문제를 찾을 수 있었다. 칩은 한 번 만들면 고칠 수 없는 만큼, 앞으로 나의 칩을 설계할 때도 시뮬레이션으로 결과를 꼼꼼히 확인하는 습관이 중요하다고 느꼈다.